C++ LANGUAGE

What is C++?

C++ is a cross-platform language that can be used to create high-performance applications.

C++ was developed by Bjarne Stroustrup, as an extension to the C language.

C++ gives programmers a high level of control over system resources and memory.

The language was updated 4 major times in 2011, 2014, 2017, and 2020 to C++11, C++14, C++17, C++20.

Why Use C++

C++ is one of the world's most popular programming languages.

C++ can be found in today's operating systems, Graphical User Interfaces, and embedded systems.

C++ is an object-oriented programming language which gives a clear structure to programs and allows code to be reused, lowering development costs.

C++ is portable and can be used to develop applications that can be adapted to multiple platforms.

C++ is fun and easy to learn!

As C++ is close to C, C# and Java, it makes it easy for programmers to switch to C++ or vice versa.

C++ User Input & OUTPUT:

`using namespace std` means that we can use names for objects and variables from the standard library.

`cout` is pronounced "see-out". Used for output, and uses the insertion operator (`<<`).

`cin` is pronounced "see-in". Used for input, and uses the extraction operator (`>>`).

What Is the C++ Namespace?

Namespace in C++ is the declarative part where the scope of identifiers like functions, the name of types, classes, variables, etc., are declared. The code generally has multiple libraries, and the namespace helps in avoiding the ambiguity that may occur when two identifiers have the same name.

C++ Datatypes:

- `int` - stores integers (whole numbers), without decimals, such as 123 or -123
- `double` - stores floating point numbers, with decimals, such as 19.99 or -19.99
- `char` - stores single characters, such as 'a' or 'B'. Char values are surrounded by single quotes
- `string` - stores text, such as "Hello World". String values are surrounded by double quotes
- `bool` - stores values with two states: true or false

C++ Data Types

```
int Num = 5;                    // Integer (whole number)
float Var= 5.99;        // Floating point number
double temp= 9.98;      // Floating point number
char ch = 'D';            // Character
bool b = true;          // Boolean
string txt= "Hello";        // String
```

Function Overloading:

With function overloading, multiple functions can have the same name with different parameters:

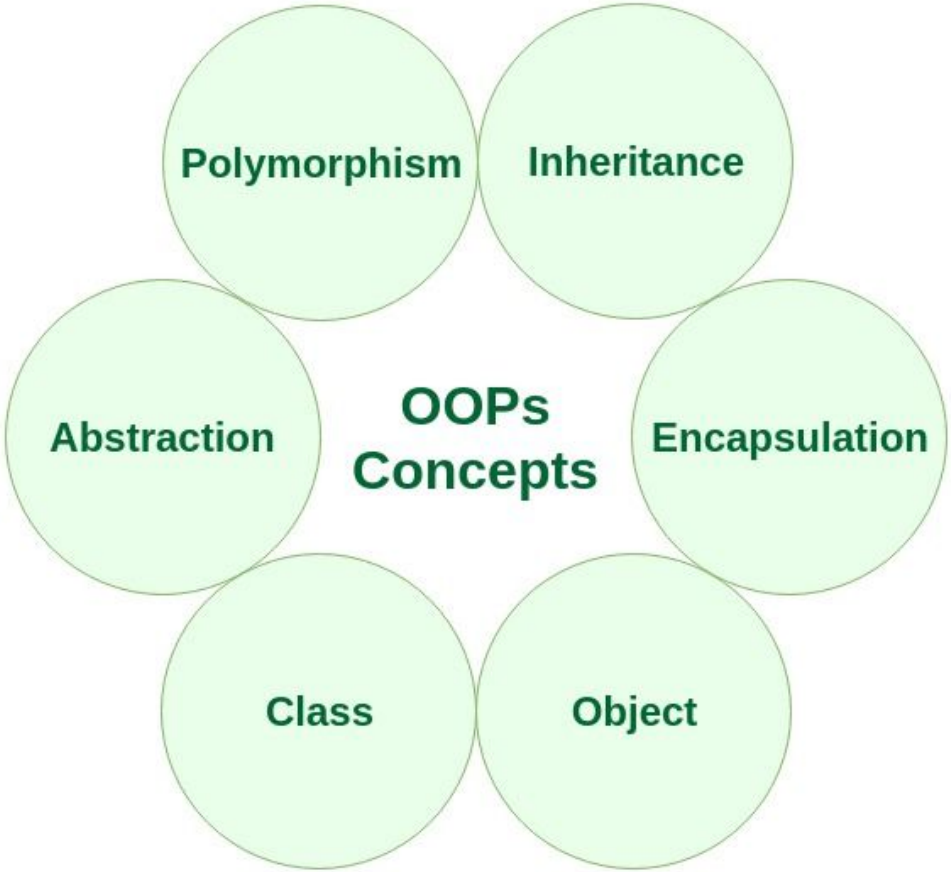
Example

```
int myFunc(int x)

float myFunc(float x)

double myFunc(double x, double y)
```

C++ What is OOP?

OOP stands for Object-Oriented Programming.

- Procedural programming is about writing procedures or functions that perform operations on the data, while object-oriented programming is about creating objects that contain both data and functions.
- Object Oriented Programming is a paradigm that provides many concepts such as inheritance, data binding, polymorphism etc.


Polymorphism
Inheritance
Abstraction
OOPs Concepts
Encapsulation
Class
Object

C++ Class Definitions:

Collection of objects is called class. It is a logical entity.

A Class in C++ is the foundational element that leads to Object-Oriented programming. A class instance must be created in order to access and use the user-defined data type data members and member functions. An object's class acts as its blueprint.

Create a Class:

To create a class, use the `class` keyword:

Example

Create a class called "`MyClass`":

```
class MyClass {        // The class
  public:              // Access specifier
    int Num;         // Attribute (int variable)
   string Text;   // Attribute (string variable)
};
```

Create an Object:

In C++, an object is created from a class. We have already created the class named `MyClass`, so now we can use this to create objects.

To create an object of `MyClass`, specify the class name, followed by the object name.

To access the class attributes (`Num` and Text), use the dot syntax (`.`) on the object:

```
class MyClass {     // The class
  public:   // Access specifier
    int myNum;     // Attribute
(int variable)
    string myString;   //
Attribute (string variable)
};
```

```
int main() {
  MyClass myObj;   // Create an
object of MyClass
 //Access attributes and set
values
  myObj.myNum = 15;
  myObj.myString = "Statement";

  // Print attribute values
  cout << myObj.myNum << "\n";
  cout << myObj.myString;
  return 0;
}
```

C++ Class Methods:

Methods are functions that belongs to the class.

There are two ways to define functions that belongs to a class:

- Inside class definition
- Outside class definition

You access methods just like you access attributes; by creating an object of the class and using the dot syntax (`.`):

- Outside class definition

To define a function outside the class definition, you have to declare it inside the class and then define it outside of the class. This is done by specifying the name of the class, followed the scope resolution `::` operator, followed by the name of the function.

C++ Constructors

A constructor in C++ is a special method that is automatically called when an object of a class is created.

It is used to initialize the data members of new object generally. The constructor in C++ has the same name as class or structure.

The Constructors prototype looks like this:

<**class**-name> (list-of-parameters);

The following syntax is used to define the class's constructor:

<**class**-name> (list-of-parameters) { // constructor definition }

The following syntax is used to define a constructor outside of a class:

<**class**-name>: :<**class**-name> (list-of-parameters){ // constructor definition}

Constructors lack a return type since they don't have a return value.

C++ Access Specifiers:

The `public` keyword is an access specifier. Access specifiers define how the members (attributes and methods) of a class can be accessed. In the example above, the members are `public` - which means that they can be accessed and modified from outside the code.

However, what if we want members to be private and hidden from the outside world?

In C++, there are three access specifiers:

- `public` - members are accessible from outside the class
- `private` - members cannot be accessed (or viewed) from outside the class
- `protected` - members cannot be accessed from outside the class, however, they can be accessed in inherited classes.

Data Abstraction in C++

Data abstraction refers to providing only essential information to the outside world and hiding their background details, i.e., to represent the needed information in program without presenting the details.

Data abstraction is a programming (and design) technique that relies on the separation of interface and implementation.

Let's take one real life example of a TV,AC, and so on.

Implementation of Abstraction in C++:

1. Using classes
2. Through header files

Benefits of Abstraction in C++:

1. Abstraction increases the reusability of the code because of the proper partitioning.

2. It reduces the complexity as well as the redundancy of the code, hence increasing the readability.

3. A programmer does not need to write the low level code.

4. Data Abstraction avoids the code duplication, i.e., programmer does not have to undergo the same tasks every time to perform the similar operation.

5. Due to abstraction, the important parts of the code are secure as only the essential features are provided to the user and they don’t know the background details.

Encapsulation:

Binding (or wrapping) code and data together into a single unit is known as encapsulation. For example: capsule, it is wrapped with different medicines.

Encapsulation is typically understood as the grouping of related pieces of information and data into a single entity. Encapsulation is the process of tying together data and the functions that work with it in object-oriented programming.

Encapsulation in C++

Class

```
class Rectangle {
  public:
    int length;
    int breadth;
    int getArea() {
      return length * breadth;
    }
  };
```

Why Encapsulation?

- It is considered good practice to declare your class attributes as private (as often as you can). Encapsulation ensures better control of your data, because you (or others) can change one part of the code without affecting other parts
- Increased security of data

Object-oriented programming has several advantages over procedural programming:

- OOP is faster and easier to execute
- OOP provides a clear structure for the programs
- OOP helps to keep the C++ code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
- OOP makes it possible to create full reusable applications with less code and shorter development time.

Friend Function:

A friend function of a class is defined outside that class scope but it has the right to access all private and protected members of the class. Even though the prototypes for friend functions appear in the class definition, friends are not member functions.

A friend can be a function, function template, member function, or a class or class template, in which case the entire class and all of its members are friends.

Declaring Friend Function

To declare a function as a friend of a class, precede the function prototype in the class definition with the keyword **friend** as follows

```
class className {

    ... .. ...

    friend returnType functionName(arguments);

    ... .. ...

 }
```

C++ Inheritance:

- In C++, inheritance is a process in which one object acquires all the properties and behaviors of its parent object automatically. In such way, you can reuse, extend or modify the attributes and behaviors which are defined in other class.
- In C++, it is possible to inherit attributes and methods from one class to another. We group the "inheritance concept" into two categories:
  - derived class (child) - the class that inherits from another class
  - base class (parent) - the class being inherited from

To inherit from a class, use the : symbol.

C++ Programming Single Inheritance Syntax

```
class A   // base class
{
    ..........
};
class B : acess_specifier A   // derived class
{
    ...........
  } ;
```

C++ Multilevel Inheritance:

- When one class inherits another class which is further inherited by another class, it is known as multi level inheritance in C++. Inheritance is transitive so the last derived class acquires all the members of all its base classes.
- A class can also be derived from one class, which is already derived from another class.

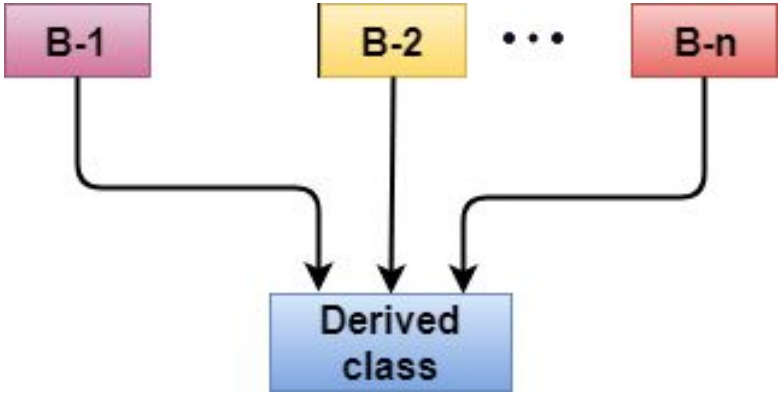
In the following example, `MyGrandChild` is derived from class `MyChild` (which is derived from `MyClass`).

SYNTAX:

```
class A {

... .. ...

};

class B: public A {

... .. ...

};

class C: public B {

... ... ...

  };
```

C++ Multiple Inheritance:

- **Multiple inheritance** is the process of deriving a new class that inherits the attributes from two or more classes.
- A class can also be derived from more than one base class, using a comma-separated list:



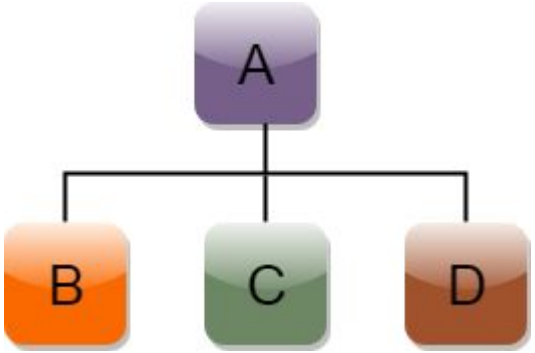
Syntax of the Derived class:

```
class base1 {
  public:
      void Function1( ) {....}
};
class base2 {
    void Function2( ) {....}
};
```

```
class derived : public base1, public base2 {

};

int main() {
    derived obj;
    obj.Function1();
   obj.Function2();
}
```

C++ Hierarchical Inheritance:

Hierarchical inheritance is defined as the process of deriving more than one class from a base class.



Syntax of Hierarchical inheritance:

```
class A
{
    // body of the class A.
}
class B : public A
{
    // body of class B.
}
class C : public A
{
    // body of class C.
}
class D : public A
{
    // body of class D.
}
```

C++ Polymorphism:

- The term "Polymorphism" is the combination of "poly" + "morphs" which means many forms. It is a greek word. In object-oriented programming.
- Polymorphism uses those methods to perform different tasks. This allows us to perform a single action in different ways.
- It simply means more than one form. That is, the same entity (function or operator) behaves differently in different scenarios.

There are two types of polymorphism in C++:

1. Compile Time
    a. Function Overloading
    b. Operator Overloading
2. Runtime
    a. Virtual Function

Operator Overloading in C++:

- In C++, we can define how operators behave for user-defined types like class and structures. For example,The `+` operator, when used with values of type `int`, returns their sum. However, when used with objects of a user-defined type, it is an error.
- In this case, we can define the behavior of the `+` operator to work with objects as well.
- This concept of defining operators to work with objects and structure variables is known as operator overloading.

Syntax for C++ Operator Overloading

The syntax for overloading an operator is similar to that of function with the addition of the `operator` keyword followed by the operator symbol.

```
returnType operator symbol
(arguments)
{
    // Code / Statements
 }
```

Here,

- `returnType` - the return type of the function
- `operator` - a special keyword
- `symbol` - the operator we want to overload (`+`, `<`, `-`, `++`, etc.)
- `arguments` - the arguments passed to the function

Example:

```
int a;
float b,sum;
sum = a + b;
```

Here, variables “a” and “b” are of types “int” and “float”, which are built-in data types. Hence the addition operator ‘+’ can easily add the contents of “a” and “b”. This is because the addition operator “+” is predefined to add variables of built-in data type only.

working/Logic behind Operator Overloading:

```
#include <iostream>
 using namespace std;

class A {

   statements;

};
```

```
int main()

{

    A a1, a2, a3;


    a3 = a1 + a2;


    return 0;

}
```

Can We Overload All Operators?

- Sizeof
- Scope resolution (::)
- Class member access operators (.(dot), .* (pointer to member operator))
- Ternary or conditional (?:)

- **Compile time polymorphism:**

The overloaded functions are invoked by matching the type and number of arguments. This information is available at the compile time and, therefore, compiler selects the appropriate function at the compile time. It is achieved by function overloading and operator overloading which is also known as static binding or early binding.

- **Runtime polymorphism**: Run time polymorphism is achieved when the object's method is invoked at the run time instead of compile time. It is achieved by method function overriding which is also known as dynamic binding or late binding.

```
class Parent {
    access_modifier :
        // overridden function
  return_type name_of_the_function()
  {
        // Code
  }
};
```

```
class child : public Parent {
    access_modifier :
        // overriding function
  return_type name_of_the_function()
  {
        // Code
  }
};
```

Virtual Function

A virtual function is a member function that is declared in the base class using the keyword virtual and is re-defined (Overridden) in the derived class.

Some Key Points About Virtual Functions:

- Virtual functions are Dynamic in nature.
- They are defined by inserting the keyword "**virtual**" inside a base class and are always declared with a base class and overridden in a child class
- A virtual function is called during Runtime

Syntax:

```
class Base {
public:
    virtual func()
    {
        // definition
    }
};
```

```
class Derived : public Base {
public:
    func()  // override
    {
        // new definition
    }
};
```

C++ Exceptions:

When executing C++ code, different errors can occur: coding errors made by the programmer, errors due to wrong input, or other random things.

When an error occurs, C++ will normally stop and generate an error message. The technical term for this is: C++ will throw an exception (throw an error).

An exception is an unexpected event that occurs during program execution. For example,

```
divide_by_zero = 7 / 0;
```

C++ *try and catch*

The ***try*** statement allows you to define a block of code to be tested for errors while it is being executed.

The ***throw*** keyword throws an exception when a problem/Error is detected

The ***catch*** - code that handles the exception thrown by the throw keyword

Syntax for Exception Handling in C++:

```
try {
    // code that may raise an exception
    throw argument;
}
catch (exception) {
    // code to handle exception
 }
```